

A Modular Python Simulator for mmWave Vehicle-to-Infrastructure Link-Layer Research

Ramanathan Subramanian
subramanian.r@northeastern.edu

Abstract—We present an open-source simulator for millimeter-wave vehicle-to-infrastructure (V2I) link-layer research. The tool integrates map-matched mobility, geometric line-of-sight channel abstraction, runtime-selectable scheduling, and two-dimensional resource-block packing (2DRBP) in a single self-contained desktop application. A deterministic `sim_engine` drives per-timestep snapshots for headless batch studies and interactive playback, while a parallel research path exposes YAML-configured city presets, dual-band (28/39 GHz) radio models, and pluggable schedulers. Session exports bundle JSON manifests, CSV trim-loss traces, and an embedded NumPy workspace for offline analysis. The design targets reproducible experimentation without external licensed runtimes or opaque native packing binaries.

Index Terms—mmWave, V2I, link-layer simulation, 2DRBP, resource allocation, reproducible research

I. Introduction

Millimeter-wave V2I systems couple urban mobility, directional beams, blockage, and time–frequency resource allocation. Link-layer studies require an end-to-end loop: vehicle positions and routes, beam and LoS state, service-block demand, packing into a normalized resource grid, and metrics such as trim loss and utilization over many timesteps. Few open tools expose this loop with both an interactive map view and machine-readable session artifacts.

This simulator closes that gap with a modular Python architecture, a PySide6 desktop GUI, and documented validation sweeps. The default path (`sim_engine`) emphasizes reproducible 2DRBP experiments; an extended core/channel/mobility stack supports comparative studies with city-scale presets and alternative schedulers.

A. Contributions

The release provides:

- A unified timestep engine with fixed-seed determinism, bundled vehicular routes, and per-step resource service block (RSB) generation.
- In-process 2DRBP packers (Guillotine, Shelf, MaxRects) with trim-loss reporting and density sweeps over vehicle counts $\{1, 2, 5, 10, 50\}$.
- A dual-path architecture: interactive `sim_engine` for teaching and prototyping, plus YAML-driven research scenarios (Boston, San Francisco, RTP presets; dual-band channel; Max-CQI, proportional-fair, latency-aware scheduling).
- Self-contained distribution: `pip install -e ".[dev,gui]"` on macOS, Windows, and Linux; optional vendored

C++ rectangle-bin sources for parity checks without a mandatory external executable.

- ZIP session export (`manifest.json`, CSV, `session.npz`) for spreadsheets, notebooks, and third-party analysis tools.

II. Simulator Architecture

The default engine maintains a `SimSession` keyed by vehicle count and random seed. Each `step()` advances mobility along precomputed routes, updates LoS probability along polylines, assigns RSB dimensions (stochastic per step so trim loss varies even at low vehicle counts), and invokes the selected packing algorithm. Snapshots feed the GUI and headless CLIs alike, ensuring identical numerics whether the user runs `-gui`, `-config`, or `-batch`.

The research path loads `scenario_*.yaml` files through a schema-validated configuration layer. Phases (warm-up, data collection) gate KPI logging; the channel module applies geometric blockage and optional codebook versus ideal beamforming modes at 28 and 39 GHz. Both paths share validation scripts that emit JSON reports under `artifacts/validation/`.

III. Mobility and Channel Models

Mobility. The GUI path uses OSM-style bundled routes with configurable vehicle counts (up to 50 in the default scenario). The research path adds open-license city presets with rectangular extents and synthetic lane grids or CSV map-matched traces.

Channel. Route-based LoS probability drives beam coloring on the aerial map. The extended channel model combines 3GPP-inspired path loss with geometric LOS/NLOS classification, dual-band carrier frequencies, and a switchable beam mode (codebook vs. ideal) for sensitivity analysis. These abstractions trade full TR 38.901 cluster models for transparency and fast iteration—appropriate for link-layer allocation studies rather than PHY waveform design.

IV. Resource Allocation and 2DRBP

Vehicles request normalized resource blocks each timestep. The packer places axis-aligned rectangles into a fixed grid; unplaced demand contributes to trim loss, the primary comparative metric across vehicle densities. Three algorithms ship in-process: Guillotine splits, Shelf rows, and MaxRects free-rectangle management—each selectable at runtime without recompilation.

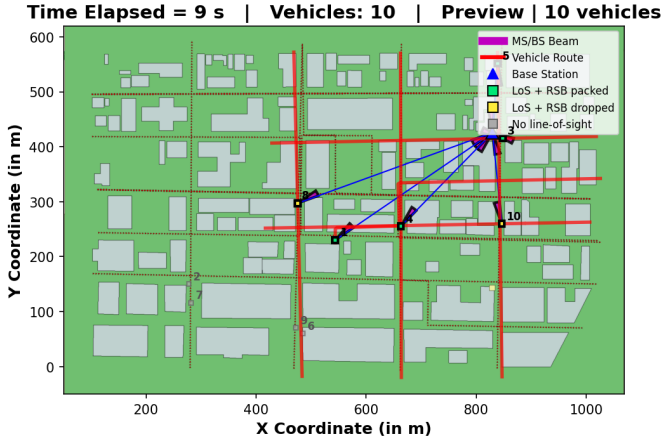


Fig. 1. Aerial map: vehicular routes, LoS beams, and packing-status coloring for active users.

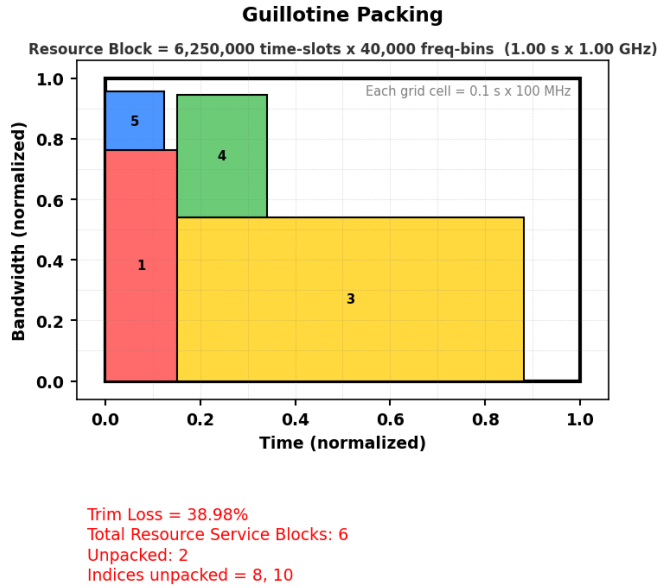


Fig. 2. Resource-block packing grid with per-step trim-loss summary.

Batch mode sweeps n_{Vehicle} over five standard counts with multiple trials per point, producing mean and standard deviation of trim loss for regression against published V2I allocation curves. Scheduler plugins on the research path allocate users before packing when studying fairness or latency policies.

V. Interactive Visualization

The desktop application couples two synchronized views: an aerial map with routes, LoS beams, and per-vehicle status colors, and a packing panel with a trim-loss readout and session-persistent trend chart. Figure 1 shows the map view during a ten-vehicle preview run; Figure 2 shows the corresponding 2DRBP placement and summary statistics.

A. Research Extensions

City-preset demos (`scripts/demo_city_presets.py`) exercise the legacy stack independently of the GUI routes. Researchers can mix presets, beam modes, and schedulers in one YAML file while the desktop app remains the fastest on-ramp for 2DRBP visualization.

VI. Reproducibility and Session Export

Fixed seeds yield bit-stable timestep sequences. `python scripts/generate_readme_assets.py` renders figure previews; `bash scripts/build_paper.sh` rebuilds this document. Completed GUI runs export a ZIP archive containing structured metadata, per-step trim-loss CSV (Excel-friendly), JSONL event logs, and `session.npz` reloadable via `load_workspace_npz()`. CI validates packing parity (including Linux-built native checks) and regenerates the PDF on paper/ changes.

VII. Conclusion

The simulator is a self-contained mmWave V2I link-layer testbed: deep enough for allocation and mobility studies, yet installable in minutes for labs and independent projects. By unifying visualization, batch sweeps, dual-band research hooks, and portable session artifacts, it lowers the cost of credible, repeatable V2I experiments without tying results to a single vendor toolchain.

Acknowledgment

Earlier open releases of mmWave V2I simulation (MathWorks File Exchange 84948) informed the problem formulation and 2DRBP metrics used here.

References

- [1] R. Subramanian, "A Resource Allocation Scheme for Multi-User MmWave Vehicle-to-Infrastructure Communication," *Future Technologies Conference (FTC)*, 2018.
- [2] R. Subramanian, "Link Layer Simulator for mmWave V2I Network," *MathWorks File Exchange*, 2019.